2018 44th Euromicro Conference on Software Engineering and Advanced Applications

Incremental Verification of Complex Event Processing Applications for System Monitoring

Andreas Metzger, Christian Reinartz, Klaus Pohl
paluno (The Ruhr Institute for Software Technology)
University of Duisburg-Essen
Essen, Germany
{*andreas.metzger, christian.reinartz, klaus.pohl*}*@paluno.uni-due.de*

***Abstract*—Complex Event Processing (CEP) facilitates monitoring large-scale, distributed systems. CEP applications analyze real-time streams of events to detect patterns that indicate problems that may require an adaptation of the running system. Like for any software system, developers may introduce faults when designing and implementing CEP applications. Such faults may imply that true problems may not be detected during systems operation, or false alarms may be raised even though no problem exists. Therefore, verifying the CEP application during design time is critical to ensure correct system monitoring at run-time. To address the scalability problem of verifying CEP applications, we propose an incremental verification approach building on recent advances in model checking. Results of an initial evaluation indicate under which assumptions our approach scales better than standard model checking.**

I. Introduction

Complex Event Processing (CEP) facilitates monitoring large-scale, distributed systems [1], [2], [3]. CEP applications are capable of collecting, aggregating and analyzing real-time streams of events to detect patterns and generate more meaningful, so called, complex events. Such complex events may indicate problems during system execution, which in turn may be handled by an adaptation of the running system [4], [5], [6], [7]. CEP was applied in areas such as cloud computing [8], business process management [9], banking and finance [10], telecommunication [11], and transport and logistics [12].

Like for any software system, developers may introduce faults when designing and implementing CEP applications. Such faults imply that true problems may not be detected during system operation, or that false alarms may be raised even though no problems exist [13]. Therefore, verifying the CEP application during design time is critical to ensure correct system monitoring at run time.

However, when using standard verification techniques, the state-space explosion problem becomes a problem. The size of the state space directly correlates with the number of events to be checked for the CEP application. Due to the potentially large number and high arrival rate of events in real-world CEP applications, this state-space explosion becomes a limiting factor.

Researchers have thus proposed dedicated verification techniques for CEP applications. The state space explosion problem is addressed in two principal ways. On the one hand, the verification process is limited to a small subset of execution paths [14]. This limitation means that the verification results may not generalize to all execution paths of a CEP application. It therefore faces the same shortcomings as if one would use dynamic testing, where the test cases only cover a subset of all potential inputs (e.g., see [15]). On the other hand, the use of bounded model checking is proposed, requiring developers to determine upper bounds on the number of events on the incoming event streams [16], [17], [18]. The challenge developers face when seeking to determine upper bounds is that it is unknown at design time how many events and which events may lead to a failure of the CEP application. We illustrate this challenge in Fig. 1.

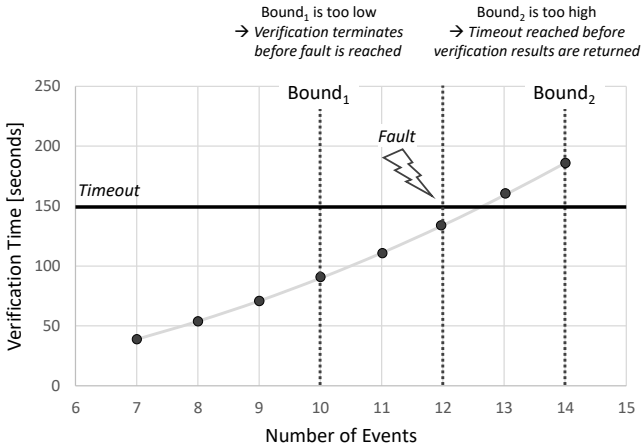

Figure 1: Consequences of setting wrong bounds when verifying CEP applications

In the figure, the bounds for the model checker are given as the maximum number of events that are checked during verification. If the bound (i.e., the number of events) is chosen too small (Bound$_1$ in the figure), verification may terminate before a fault is uncovered. If the bound is chosen too large (Bound$_2$ in the figure), verification may not

978-1-5386-7383-6/18/$31.00 ©2018 IEEE
DOI 10.1109/SEAA.2018.00055

Authorized licensed use limited to: Technische Hochschule Ingolstadt. Downloaded on March 13,2023 at 12:46:33 UTC from IEEE Xplore. Restrictions apply.

terminate in reasonable time or it will reach a predefined timeout before verification results can be returned.

This paper proposes an *incremental* verification approach, which avoids the need to determine effective bounds. Incremental verification means that we iteratively increase the verification bounds and verify the system in each iteration. To achieve scalability of the incremental approach, we propose exploiting incremental model checking. In particular, we use a specific form of incremental model checking, which was introduced recently, in which the model remains constant but the bounds are incrementally increased (e.g., see [19], [20]).

Incremental verification typically requires less time for each verification step compared to a complete re-verification. We provide results of an initial evaluation, indicating under which performance characteristics of the underlying incremental model checker our approach for CEP verification scales better than when using a standard model checker. Based on existing benchmark measurements for incremental model checker performance, our results suggest that our approach may provide better scalability in 70% of cases.

Section II discusses related work. Section III describes the principal solution idea for incremental verification. Section IV presents the initial evaluation results. Section V concludes.

II. Related Work

Different directions for verifying CEP applications were explored in the literature. Rabinovic et al. propose a *hybrid approach* for behavior analysis of the CEP applications [16]. They use model checking to derive test case scenarios from a CEP application model. Test case scenarios are used for simulation to determine whether the CEP application behaves correctly. Due to the focus on selected test case scenarios, the approach does not generalize to all execution paths of a CEP application (see Section I).

Ericsson et al. use *model checking* to verify the CEP application [17]. The CEP application is modeled using REX, a graphical modeling tool for event processing networks and verified with an extension of the UPPAAL model checker. Their approach requires developers to determine upper bounds on the number of events to scale in practice.

Weidlich et al. use *colored petri net models* to verify CEP applications [18]. Events are produced randomly by a token generator. For the analysis of the CEP system, the number of events generated by the token generator is bound. They propose using domain knowledge to determine upper bounds. However, as the place of faults is not known (see Section I), domain knowledge may not help to uncover all faults.

Cugola et al. use *constraint solving* to analyze the behavior of CEP applications [14]. They decompose a given property into constraint solving problems, which are verified individually. If at least one event sequence exists that satisfies the constraints, the property is considered satisfied. This approach can prove if a property is satisfied by at least one event sequence. Yet, it does generalize to all possible event sequences.

In our own previous work, we used *reachability analysis* to detect deadlocks in CEP applications [21]. Similar to [22], we formalized the CEP applications as petri net models and used off-the-shelf petri net analyzers. As a follow-up work, we used *model checking*, in particular Tapaal, delivering significant performance improvements. However, we did not yet use an incremental approach for verification and thus use our Tapaal implementation as baseline for comparison in Section IV.

III. Verification Approach

To address the problem of determining effective upper bounds for verifying CEP applications, we propose an incremental verification approach. Incremental verification means that bounds are iteratively increased. For each increment, a model of the CEP application is checked. Two ways of realizing this incremental verification are:

Naïve approach. A naïve way is to use a standard model checker and re-run the model checker for progressively larger bounds. Such a naïve incremental verification approach would start with bound 1 and increase the bounds until a timeout for the single model checking task is reached, or until a total timeout (cumulative time of all model checking tasks) is reached. The naïve approach faces obvious scalability issues. As the model is (re-)verified completely for each iteration, many redundant checks are done by the model checker.

Advanced approach. To ensure scalability of incremental verification, we suggest exploiting incremental model checking technology. Incremental model checking reuses previous verification results to deliver sub-sequent verification results much faster than if rerunning the verification from scratch. Two variants of incremental model checking techniques exist: (1) Incremental model checking may be applied in order to verify changes in the model of the system. For example, Molnar et al. modularize the model checking task into a set of smaller tasks [23]. If one part of the model is changed, these smaller verification tasks can be selectively re-executed rather than verifying the complete model. (2) Incremental model checking may be applied to automatically increase the state space (i.e., bounds) to be verified. For example, Günther et al. introduce an incremental approach for verifying loops in program code [20]. To this end, loops are incrementally unfolded and verified.

The advanced verification approach exploits the second variant of incremental model checking. Previous verification results on the same model, but with different bounds, are reused to deliver subsequent verification results much faster than if rerunning the verification from scratch. In contrast to the work presented in [20], this means finding a way

Authorized licensed use limited to: Technische Hochschule Ingolstadt. Downloaded on March 13,2023 at 12:46:33 UTC from IEEE Xplore. Restrictions apply.

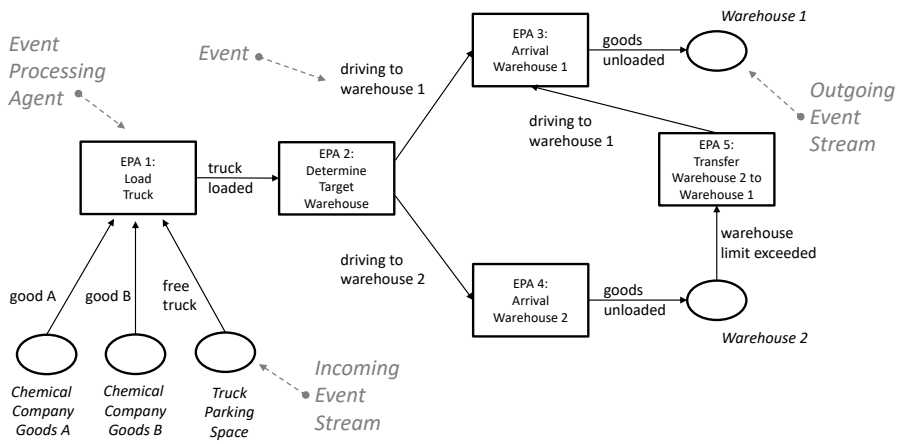

Figure 2: CEP application for monitoring and managing a distributed transport and logistics system

of encoding a model of the CEP application as input to the model checker. To this end, we may exploit previous approaches, such as the ones presented in [21] and [22]. However, the encoding must be chosen in such a way that the model checker can incrementally "unfold" and verify the model with respect to the number of events that are checked.

We expect that this advanced realization of incremental verification scales better than the naïve verification approach. We support this intuition by the initial experimental results presented below.

IV. Initial Evaluation

We aim to answer whether the advanced approach in principle scales better than the naïve approach. We use an example CEP application as basis for evaluation.

A. Example CEP Application

We briefly introduce background on CEP to understand the example that follows. A CEP applications is realized by means of event processing networks, which receive streams of incoming, basic events and generate streams of outgoing, complex events. Event processing networks typically contain several event processing agents (EPAs). EPAs offer functions such as filter, aggregation, pattern detect, and transformation [2].

Our example represents a CEP application used to monitor and manage a distributed transport and logistics system and is shown in Fig. 2. The system monitors and manages trucks that transport containers of dangerous chemical goods from a chemical company to two warehouses. Each truck can load one container of goods either of class A or class B and deliver them to Warehouse 1 or Warehouse 2. Warehouse 2 has a limited capacity and can only hold 30 containers. One problem that may be monitored during operations is that the capacity of Warehouse 2 is reached. In such a case, overflow containers may be transferred from Warehouse 2 to Warehouse 1. An important safety property is defined due to handling of dangerous goods: Only goods of the same class may be stored at one warehouse at the same point in time.

The CEP application consists of five EPAs.[1] EPA 1 recognizes the load status of a truck. If a container of goods of class A or B (represented by events *good A* and *good B*) is ready to be transported and a truck is available (represented by event *free truck*), the CEP application assigns the container to this free truck. It then generates a new complex event (*truck loaded*), consumed by EPA 2. EPA 2 determines to which warehouse the truck should drive. Due to the restriction that only one class of goods may be stored at one warehouse at the same point in time, EPA 2 takes all previous *truck loaded* events into account. Depending on the decision of EPA 2, the respective *driving to warehouse* event is sent to EPA 3 or EPA 4, which wait for the arrival of the truck. Upon arrival, the truck is unloaded and the goods are placed in the warehouse (*goods unloaded* events). If the containers at Warehouse 2 exceed the storage capacity, EPA 5 receives the *warehouse limit exceeded* event, which in turn dispatches a local truck to transport the overflow container to Warehouse 1.

Under certain circumstances, a failure may occur in this event processing network. When the limit of Warehouse 2 exceeds 30 containers, goods are offloaded to Warehouse 1. This leads to a failure if, for example, Warehouse 1 stores goods of class A and Warehouse 2 stores goods of class B. The fault in the CEP application is that EPA 5 does not check if the target Warehouse 1 stores goods of a different class than the origin Warehouse 2. This violates the aforementioned safety property, i.e., that one warehouse never stores the same types of goods at the same time. This fault means that the violation of this safety property would never be monitored and thus would go unnoticed during system operation.

[1]Even no shown in the figure, each EPA receives all necessary real world events, such as positions of the trucks, job confirmations from drivers, etc.

Authorized licensed use limited to: Technische Hochschule Ingolstadt. Downloaded on March 13,2023 at 12:46:33 UTC from IEEE Xplore. Restrictions apply.

B. Experimental Results

Verification of CEP applications with standard model checkers has been investigated in the literature (see Section II). As a baseline for our experiments, we employ the approach presented in [21]. There, CEP applications are encoded as petri net models. We use these petri net models as input for the standard model checker Tapaal [24]. Tapaal is a bounded model checker, which means that upper bounds on the size of the state space can be provided as parameter of the model checker. We use Tapaal 3.2.0, running as a single process on an Intel Xeon Processor with 32 GB of RAM running 64-bit Ubuntu Linux in version 14.04 LTS.

For each of the bounds, we measure the verification time of Tapaal. Based on these individual measurements, we aggregate the computation times for the naïve approach. These cumulative verification times are shown in Fig. 3 in black. As can be seen, the naïve approach faces significant scalability issues, with verification times increasing exponentially.

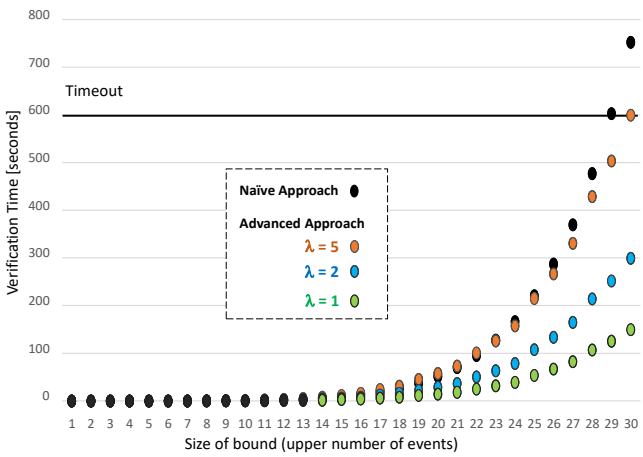

Figure 3: Verification times for different realizations of incremental approach

As an example, if we assumed a timeout of 600 seconds (10 minutes), which may be tolerable for developers during design time, the naïve approach would only be able to check 28 events. However, in our example, the fault only manifests itself after 30 containers, and thus 30 incoming *goods* events.

To assess the scalability of the advanced approach, we use results reported by Günther et al. [20] on the performance of incremental model checkers. They have used existing benchmarks (for bit vector problems) and evaluated them with an incremental model checking algorithm built into existing solvers. Günther et al. executed 32 benchmarks using 6 different model checkers. In particular they measured the time, t_{inc}, that incremental model checking requires to incrementally check all bounds up to a certain bound b. They compared this with time, t_{std}, that a standard model checker takes to only check bound b. For each of the results, they computed a factor $\lambda = (t_{\text{inc}} - t_{\text{std}})/t_{\text{std}}$ that expresses the relative differences in execution times.

The λ values measured in [20] ranged from 1 to 54.9, with 30% having a $\lambda \leq 1$, 70% a $\lambda \leq 2$, and 80% a $\lambda \leq 5$. We thus use $\lambda = 1$, 2 and 5 respectively to show the potential speedup of our advanced approach over the naïve approach. As seen in Fig. 3 (in green, blue and orange), the advanced approach scales visibly better for $\lambda \leq 2$. It thereby allows verifying the CEP application for a much higher number of events, and thereby would be able to catch the 30 container fault from above.

V. Conclusion and Perspectives

We presented the concepts of an incremental approach for verifying CEP applications to detect faults in the CEP application during design time, thereby preventing false monitoring results at run time. As future work, we will implement the approach by adapting the implementation of existing incremental model checkers such that they can directly cope with event processing networks as input. We also plan performing further experiments using different CEP applications.

Acknowledgment: We thank the anonymous reviewers for their constructive comments. The research leading to these results received funding from the European Union's Seventh Framework Programme FP7/2007-2013 under grant agreement 610802 (CloudWave), from the European Unions Horizon 2020 research and innovation programme under grant agreement no. 731932 (TransformingTransport), as well as from the EFRE co-financed operational program NRW.Ziel2 under grant agreement 005-1010-0012 (LoFIP).

References

[1] R. Rabiser, S. Guinea, M. Vierhauser, L. Baresi, and P. Grünbacher, "A comparison framework for runtime monitoring approaches," *Journal of Systems and Software*, vol. 125, pp. 309–321, 2017.

[2] O. Etzion, P. Niblett, and D. C. Luckham, *Event processing in action*. Manning Greenwich, 2011.

[3] D. Luckham, *The power of events*. Addison-Wesley Reading, 2002, vol. 204.

[4] H. Eichelberger, C. Qin, and K. Schmid, "From resource monitoring to requirements-based adaptation: An integrated approach," in *8th Int'l Conference on Performance Engineering, ICPE 2017, L'Aquila, Italy, April 22-26, 2017*. ACM, 2017.

[5] J. Simmonds, S. Ben-David, and M. Chechik, "Monitoring and recovery for web service applications," *Computing*, vol. 95, no. 3, pp. 223–267, 2013.

[6] A. Bucchiarone, C. Cappiello, E. Di Nitto, S. Gorlatch, D. Mailänder, and A. Metzger, "Design for self-adaptation in service-oriented systems in the cloud," in *European Research Activities in Cloud Computing*. Cambridge Scholars Publishing, 2012.

Authorized licensed use limited to: Technische Hochschule Ingolstadt. Downloaded on March 13,2023 at 12:46:33 UTC from IEEE Xplore. Restrictions apply.

[7] E. Di Nitto, C. Ghezzi, A. Metzger, M. P. Papazoglou, and K. Pohl, "A journey to highly dynamic, self-adaptive service-based applications," *Autom. Softw. Eng.*, vol. 15, no. 3-4, pp. 313–341, 2008.

[8] G. Aceto, A. Botta, W. de Donato, and A. Pescapè, "Cloud monitoring: A survey," *Computer Networks*, vol. 57, no. 9, pp. 2093–2115, 2013. [Online]. Available: https://doi.org/10.1016/j.comnet.2013.04.001

[9] F. Koetter and M. Kochanowski, "A model-driven approach for event-based business process monitoring," *Inf. Syst. E-Business Management*, vol. 13, no. 1, pp. 5–36, 2015.

[10] A. Artikis and others, "A prototype for credit card fraud management: Industry paper," in *11th Int'l Conference on Distributed and Event-based Systems, DEBS 2017, Barcelona, Spain, June 19-23, 2017*. ACM, 2017.

[11] A. Hakiri and A. S. Gokhale, "Data-centric publish/subscribe routing middleware for realizing proactive overlay software-defined networking," in *10th Int'l Conference on Distributed and Event-based Systems, DEBS '16, Irvine, CA, USA, June 20 - 24, 2016*. ACM, 2016.

[12] Z. Feldman, F. Fournier, R. Franklin, and A. Metzger, "Proactive event processing in action: a case study on the proactive management of transport processes (industry article)," in *7th Int'l Conference on Distributed Event-Based Systems, DEBS '13, Arlington, TX, USA - June 29 - July 03, 2013*. ACM, 2013.

[13] M. Völz, B. Koldehofe, and K. Rothermel, "Supporting strong reliability for distributed complex event processing systems," in *13th Int'l Conference on High Performance Computing & Communication, HPCC 2011, Banff, Alberta, Canada, September 2-4, 2011*. IEEE, 2011.

[14] G. Cugola, A. Margara, M. Pezzè, and M. Pradella, "Efficient analysis of event processing applications," in *9th Int'l Conference on Distributed Event-Based Systems, DEBS '15, Oslo, Norway, June 29 - July 3, 2015*. ACM, 2015.

[15] I. Ari, E. Olmezogullari, and H. Sözer, "Application of combinatorial testing techniques in complex event processing engines," in *20th Signal Processing and Communications Applications Conference, SIU 2012, Mugla, Turkey, April 18-20, 2012*. IEEE, 2012.

[16] E. Rabinovich, O. Etzion, S. Ruah, and S. Archushin, "Analyzing the behavior of event processing applications," in *Fourth Int'l Conference on Distributed Event-Based Systems, DEBS 2010, Cambridge, United Kingdom, July 12-15, 2010*. ACM, 2010.

[17] A. Ericsson, P. Pettersson, M. Berndtsson, and M. Seiriö, "Seamless formal verification of complex event processing applications," in *First Int'l Conference on Distributed Event-Based Systems, DEBS 2007, Toronto, Ontario, Canada, June 20-22, 2007*, vol. 233. ACM, 2007.

[18] M. Weidlich, J. Mendling, and A. Gal, "Net-based analysis of event processing networks - the fast flower delivery case," in *34th Int'l Conference on the Application and Theory of Petri Nets and Concurrency, PETRI NETS 2013, Milan, Italy, June 24-28, 2013*, ser. LNCS, vol. 7927. Springer, 2013.

[19] P. Schrammel, D. Kroening, M. Brain, R. Martins, T. Teige, and T. Bienmüller, "Incremental bounded model checking for embedded software," *Formal Asp. Comput.*, vol. 29, no. 5, pp. 911–931, 2017.

[20] H. Günther and G. Weissenbacher, "Incremental bounded software model checking," in *2014 Int'l Symposium on Model Checking of Software, SPIN 2014, San Jose, USA, July 21-23, 2014*. ACM, 2014.

[21] C. Reinartz, A. Metzger, and K. Pohl, "Model-based verification of event-driven business processes," in *9th Int'l Conference on Distributed Event-Based Systems, DEBS '15, Oslo, Norway, June 29 - July 3, 2015*. ACM, 2015.

[22] H. Tran and U. Zdun, "Event actors based approach for supporting analysis and verification of event-driven architectures," in *17th Int'l Enterprise Distributed Object Computing Conference, EDOC 2013, Vancouver, BC, Canada, September 9-13, 2013*. IEEE Computer Society, 2013.

[23] V. Molnár, D. Darvas, A. Vörös, and T. Bartha, "Saturation-based incremental LTL model checking with inductive proofs," in *21st Int'l Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2015, London, UK, April 11-18, 2015*, ser. LNCS, vol. 9035. Springer, 2015.

[24] J. Byg, K. Y. Jørgensen, and J. Srba, "TAPAAL: editor, simulator and verifier of timed-arc petri nets," in *7th Int'l Symposium on Automated Technology for Verification and Analysis, ATVA 2009, Macao, China, October 14-16, 2009*, ser. LNCS, vol. 5799. Springer, 2009.

Authorized licensed use limited to: Technische Hochschule Ingolstadt. Downloaded on March 13,2023 at 12:46:33 UTC from IEEE Xplore. Restrictions apply.